

A Configurable Multi-Lane FPU Supporting Exponent/Mantissa Scaling on FPGA and ASIC

Jesus Huertas-Rojas, Andrea García-Borges, Brayan Solís-Rojas, Milagro Rojas-Sánchez,
Emilio Soto-Porras, Erick Obregón-Fonseca[✉], Luis G. León-Vega[✉], Jorge Castro-Godínez[✉]

Instituto Tecnológico de Costa Rica, San Carlos, Costa Rica

Corresponding: {jocastro,l.leon}@tec.ac.cr

Abstract—Este trabajo presenta el diseño e implementación de una unidad de punto flotante de 16 bits totalmente parametrizable con replicación de carriles vectoriales para computación energéticamente eficiente. La arquitectura permite la configuración independiente de los anchos de mantisa y exponente, así como un ancho de datos escalable mediante la duplicación automática de carriles. La unidad se integró y evaluó en una plataforma AMD Kria KV260, donde demostró un bajo consumo de recursos y un consumo dinámico de energía mínimo, manteniendo la corrección funcional para la suma y la multiplicación en múltiples configuraciones de precisión. Los resultados experimentales muestran que el diseño parametrizable logra características competitivas de área y energía en comparación con una línea base de precisión fija, a la vez que ofrece una flexibilidad significativamente mayor. Si bien la frecuencia máxima alcanzada está limitada por el enrutamiento a nivel de sistema en lugar de la ruta de datos de punto flotante, la arquitectura sigue siendo adecuada para sistemas embebidos y de bajo consumo. El diseño propuesto proporciona una base viable para futuros trabajos en aritmética de precisión ajustable y aceleración específica del dominio.

Index Terms—floating point unit, parameterizable architecture, low power design, vector processing, RISC-V integration, FPGA implementation

I. INTRODUCCIÓN

El cómputo en punto flotante de precisión media se ha convertido en un componente central de muchas aplicaciones modernas, en particular en aprendizaje profundo, visión por computadora y procesamiento de señales en dispositivos embebidos. En estos entornos, el presupuesto de energía y de área suele ser muy restringido, por lo que resulta atractivo utilizar formatos compactos como *float16* y explotar el paralelismo de datos mediante unidades vectoriales especializadas. Sin embargo, incluso los formatos estandarizados, como IEEE 754 de media precisión, pueden resultar sobredimensionados para ciertas cargas de trabajo, mientras que son insuficientes para otras, lo que motiva la búsqueda de mecanismos de precisión ajustable.

Las unidades de punto flotante tradicionales, diseñadas para un único formato fijo, presentan limitaciones importantes en escenarios de cómputo heterogéneo. Por un lado, obligan a utilizar el mismo ancho de exponente y de mantisa para todas las operaciones, lo que puede derivar en un consumo de energía y de área mayor al necesario cuando la aplicación tolera errores numéricos moderados. Por otro lado, dificultan adaptar la arquitectura a dominios donde se requiere una mayor

dinámica o precisión localizada, como en etapas específicas de redes neuronales o en algoritmos numéricos sensibles a la cancelación. En plataformas FPGA, estas restricciones se agravan cuando el diseño no aprovecha de forma explícita los bloques DSP ni permite explorar el compromiso entre recursos, frecuencia máxima y precisión numérica.

En este contexto, surge la oportunidad de diseñar unidades de punto flotante con *precisión ajustable*, donde los anchos de mantisa y exponente puedan parametrizarse, y el datapath pueda replicarse automáticamente en función del ancho total de dato disponible. De esta forma, una misma unidad puede operar como FPU escalar, como unidad de dos líneas *float16x2* o incluso como arreglo de mayor paralelismo, manteniendo compatibilidad con procesadores RISC-V de 32 y 64 bits. Además, la integración explícita de celdas DSP de las FPGAs de AMD permite mejorar el desempeño y reducir el uso de LUTs frente a implementaciones puramente lógicas, lo cual es especialmente relevante para arquitecturas orientadas a aceleración de IA y DSP.

Sobre esta motivación, este trabajo presenta el diseño, implementación e integración de una unidad de punto flotante parametrizable orientada a procesadores RISC-V. La arquitectura propuesta admite el ajuste explícito de los anchos de mantisa y exponente, lo que permite explorar configuraciones de precisión adaptadas a distintos requisitos de área, energía y calidad numérica, manteniendo compatibilidad con los mecanismos estándar de extensión del ISA.

La unidad está organizada en múltiples carriles capaces de operar sobre datos empaquetados, de modo que el paralelismo efectivo se adapta automáticamente al ancho total de palabra del procesador. Este mecanismo de replicación estructural habilita la ejecución simultánea de operaciones como suma, multiplicación y recíproco en formatos compactos de 16 bits, y puede escalar hacia configuraciones de mayor densidad sin modificar el RTL base.

Para validar la propuesta, se integró la arquitectura parametrizable en un procesador RISC-V sintetizable y se evaluó sobre la plataforma AMD Kria KV260. El estudio incluye análisis de utilización de recursos lógicos y bloques DSP, frecuencia máxima alcanzable, latencia por operación y estimaciones de potencia derivadas de la herramienta de análisis de consumo. Los resultados experimentales se comparan con una versión no parametrizada y con variantes que

prescinden del uso de bloques DSP, a fin de cuantificar los beneficios de la parametrización y del paralelismo por carriles.

II. ANTECEDENTES Y TRABAJOS RELACIONADOS

El estándar IEEE 754-2019 define la representación y el comportamiento de la aritmética en punto flotante empleada en la mayoría de los procesadores modernos, incluyendo los formatos binary32, binary64 y variantes de precisión reducida como binary16 [1]. Estos formatos compactos han adquirido gran relevancia debido a su menor costo en almacenamiento, la reducción del ancho de banda de memoria y la simplificación del hardware aritmético. En particular, el formato de media precisión ha demostrado importantes beneficios en cargas de trabajo basadas en aprendizaje profundo, donde el entrenamiento en precisión mixta reduce el consumo energético y aumenta el rendimiento sin afectar la exactitud global [2]. Esta tendencia ha impulsado el desarrollo de unidades aritméticas capaces de ajustar dinámicamente la precisión para balancear exactitud, energía y área.

La arquitectura RISC-V incorpora un conjunto modular de extensiones para aritmética en punto flotante, entre ellas las extensiones F, D y Q, así como la extensión Zfh orientada a operaciones en media precisión [3]. Dichas extensiones especifican los formatos de los operandos, el manejo de excepciones, los modos de redondeo y las reglas de integración de la unidad de punto flotante dentro del pipeline del procesador. Gracias a su naturaleza abierta y extensible, RISC-V permite incorporar unidades aritméticas personalizadas o formatos no estándar manteniendo compatibilidad con el ecosistema existente. Implementaciones avanzadas, como el núcleo fuera de orden BOOM, demuestran la viabilidad de integrar tuberías FP parametrizables dentro de procesadores sintetizables y de propósito general [4].

Diversos trabajos han estudiado técnicas de cómputo aproximado y de precisión variable como mecanismos efectivos para reducir significativamente el consumo energético en aplicaciones intensivas en punto flotante. Investigaciones tempranas identificaron que reducir la precisión de manera controlada permite disminuir la energía consumida sin comprometer los requisitos de calidad de resultado [5]. Más recientemente, las arquitecturas transprecision [6] han introducido formatos configurables —como binary8 y variantes alternativas de binary16— junto con soporte hardware que permite seleccionar el formato adecuado según las necesidades de cada etapa del cálculo. Estos estudios muestran que una fracción considerable de las operaciones puede ejecutarse con menor precisión, reduciendo el tiempo de ejecución, el tráfico de memoria y la energía consumida.

El paralelismo a nivel de dato constituye otro elemento fundamental en la evolución de las unidades de punto flotante. En arquitecturas SIMD y vectoriales, los registros anchos se dividen en subpalabras que pueden procesarse en paralelo, lo cual incrementa el rendimiento y mejora la eficiencia energética. Este enfoque es empleado en la Extensión Vectorial de RISC-V y en aceleradores modernos como las TPU de Google, que se basan extensivamente en cómputo de precisión

reducida para maximizar el paralelismo. Formatos empaquetados como *float16x2* permiten ejecutar múltiples operaciones FP dentro de una misma palabra, facilitando la escalabilidad del datapath y simplificando el diseño de hardware replicado por carriles.

Finalmente, múltiples generadores de hardware han propuesto unidades FP parametrizables que sirven como base para arquitecturas eficientes y adaptables. La librería HardFloat ofrece componentes FP sintetizables y configurables, mientras que frameworks como FloPoCo generan operadores personalizados que exploran distintos compromisos entre área, latencia y precisión. Adicionalmente, estudios orientados a FPGA han mostrado cómo el uso de bloques DSP puede reducir notablemente el costo de las operaciones de multiplicación y habilitar pipelines flexibles y escalables. Este cuerpo de trabajos establece las bases conceptuales y metodológicas para el diseño de unidades FP configurables en precisión, replicables por carriles y compatibles con estándares modernos, lo cual motiva directamente la arquitectura propuesta en este artículo.

III. ARQUITECTURA DE LA FPU PARAMETRIZABLE

A. Objetivos de diseño

La unidad propuesta se concibió como un bloque aritmético reutilizable para distintos sistemas RISC-V de 32 y 64 bits, con énfasis en tres metas principales. En primer lugar, se busca escalabilidad en términos de ancho de palabra: el mismo código RTL debe poder instanciarse como unidad escalar para RV32, como unidad de dos líneas tipo *float16x2* y, en general, como motor vectorial con un número arbitrario de carriles mientras el ancho total del registro lo permita. En segundo lugar, la FPU expone de forma explícita parámetros de ancho de exponente y de fracción, permitiendo explorar configuraciones de precisión reducida más allá del formato IEEE binary16, manteniendo siempre un bit de signo. Finalmente, se persigue una arquitectura modular donde las operaciones de suma, multiplicación y recíproco se implementan como subunidades independientes, de modo que sea sencillo sustituir o mejorar cada operador sin afectar al resto del datapath.

En la implementación actual, el prototipo se sintetiza sobre la plataforma AMD Kria KV260 empleando únicamente lógica programable (LUTs, *carry chains* y multiplexores F7) para las rutas críticas, dejando la explotación de bloques DSP como optimización futura. Los informes de Vivado muestran que el bloque `fp16_vec_alu_0` utiliza del orden de mil LUTs y que las rutas de tiempo más restrictivas pertenecen a la lógica de interconexión (módulos AXI-GPIO), lo que confirma que la FPU parametrizable no se convierte en el camino crítico del sistema completo.

B. Parametrización del datapath en punto flotante

El núcleo de la propuesta es un datapath en punto flotante descrito mediante parámetros de síntesis. El ancho total del dato se define como

$$FP_W = 1 + EXP_W + FRAC_W,$$

donde EXP_W y $FRAC_W$ son parámetros enteros que especifican el número de bits del exponente y de la fracción, respectivamente, y el bit adicional corresponde al signo. De esta forma, el mismo diseño puede instanciarse con una configuración compatible con binary16 (por ejemplo, $EXP_W=5$, $FRAC_W=10$) o con formatos alternativos (p.ej., exponente más pequeño y fracción mayor) simplemente modificando los parámetros en el momento de la síntesis.

La parametrización impacta de manera directa en todas las etapas clásicas de la aritmética en punto flotante. El cálculo del sesgo del exponente y la detección de *overflow/underflow* se derivan de EXP_W , mientras que la normalización y el redondeo dependen del tamaño de la fracción. El hardware de alineamiento de operandos en el sumador se dimensiona para el máximo desplazamiento posible entre exponentes, y la lógica de normalización posterior a suma o multiplicación se implementa de forma genérica mediante contadores de ceros a la izquierda sobre la mantisa. En términos de recursos, reducir $FRAC_W$ disminuye el número de LUTs y niveles de lógica en el camino crítico, a costa de aumentar el error de redondeo; por el contrario, incrementar la mantisa mejora la precisión numérica pero incrementa el área y la latencia combinacional.

C. Mecanismo de replicación por carriles

Para explotar el paralelismo a nivel de dato, la FPU se organiza como una ALU vectorial compuesta por varios carriles (*lanes*) idénticos. El número de carriles se controla mediante el parámetro N_LANES , y el ancho total del vector se define como

$$VEC_W = N_LANES \times FP_W.$$

En configuraciones típicas, un sistema RV32 con formato de 16 bits por elemento emplea $N_LANES=2$, mientras que un sistema de 64 bits podría instanciar cuatro carriles para procesar cuatro valores de 16 bits en paralelo.

El mecanismo de replicación se implementa mediante un *generate* estructural en Verilog/SystemVerilog que divide los buses de entrada a y b en subpalabras de ancho FP_W y conecta cada subpalabra a una instancia independiente del datapath escalar. Cada carril incluye su propio sumador, multiplicador y unidad de recíproco, y produce un resultado de FP_W bits que luego se reempaqueta en el vector de salida res . No existe dependencia de datos entre carriles: todos comparten la misma señal de operación op_sel , pero operan sobre pares de operandos distintos, siguiendo un esquema de ejecución de tipo *SIMD por elemento*. Esta organización permite que las pruebas se realicen con distintos valores por carril —como se muestra en el *testbench*— y simplifica la escalabilidad del diseño a anchos de palabra mayores.

D. Operaciones soportadas

La unidad vectorial soporta tres operaciones principales: suma, multiplicación y recíproco en punto flotante. Todas ellas se implementan como datapaths combinacionales por carril, gobernados por el código de operación op_sel . La suma realiza alineamiento de exponentes, suma o resta de mantisas con soporte de *carry chains* ($CARRY8$) y posterior normalización y

redondeo al formato parametrizado. La multiplicación calcula el exponente resultante a partir de la suma de exponentes sesgados y obtiene el producto de mantisas, seguido de una etapa de normalización. En la versión actual, ambas operaciones han sido verificadas exhaustivamente mediante vectores de prueba, confirmando su correcto funcionamiento para distintas configuraciones de EXP_W y $FRAC_W$.

La operación de recíproco (OP_RCP) se implementa como una unidad aproximada, pensada como primer prototipo. A alto nivel, la mantisa de entrada se transforma mediante una aproximación inicial basada en una tabla discreta o en una función algebraica sencilla, y posteriormente se ajusta mediante una o más iteraciones de refinamiento. Esta unidad permite evaluar el impacto que tendría disponer de recíprocos en hardware dentro de la misma arquitectura parametrizable, aunque en la implementación actual se han observado pequeñas discrepancias numéricas en ciertos rangos de entrada. La corrección y robustecimiento de este operador se deja como trabajo futuro, mientras que la infraestructura de parametrización y replicación por carriles permanece válida.

E. Integración en sistemas RISC-V

Aunque la FPU parametrizable está descrita de forma agnóstica al núcleo de procesamiento, su interfaz está pensada para integrarse de manera natural en sistemas RISC-V de 32 y 64 bits. El bloque vectorial expone un conjunto reducido de señales de control (RST , ENA , op_sel) y buses de datos de ancho VEC_W , lo que permite conectarlo tanto como unidad funcional dentro del pipeline de coma flotante como acelerador acoplado mediante un puerto de extensión o un periférico de memoria mapeada.

En una configuración de integración directa con un núcleo RV32F o RV64F, cada instrucción de punto flotante podría activar la unidad con un código de operación específico, reutilizando el archivo de registros de coma flotante para suministrar y recolectar operandos en formato empaquetado. Alternativamente, en plataformas como la AMD Kria KV260 es posible integrar la FPU como un acelerador en la lógica programable conectado a través de una interfaz AXI-GPIO, de modo que un procesador maestro (ya sea un núcleo RISC-V blando o el procesador ARM del sistema) configure la operación, escriba los operandos empaquetados y lea los resultados. Esta flexibilidad permite evaluar la arquitectura propuesta tanto en entornos de investigación de microarquitectura como en prototipos de sistema heterogéneo, sin modificar el código RTL de la unidad de punto flotante.

IV. METODOLOGÍA EXPERIMENTAL

A. Implementación en FPGA

La evaluación principal se realizó sobre la plataforma AMD Kria KV260, basada en un dispositivo Zynq UltraScale+ ZU3EG. El diseño se sintetizó e implementó en Vivado 2023.2 utilizando un flujo RTL estándar y una única restricción de periodo de 10 ns (100 MHz). No se aplicaron optimizaciones manuales adicionales.

Para estudiar el impacto de la parametrización, la FPU se instanció con el formato binary16 estándar (1 bit de signo, 5 bits de exponente y 10 bits de fracción), así como con configuraciones alternativas de precisión reducida. Las operaciones de suma y multiplicación fueron verificadas correctamente en todas las configuraciones evaluadas; la operación de recíproco funciona funcionalmente pero presenta errores numéricos en ciertos rangos, por lo que se considera trabajo futuro.

La utilización de recursos para la configuración FP16 con dos carriles ($N_LANES=2$) fue de 1092 LUTs, 92 celdas CARRY8 y 21 multiplexores F7, sin empleo de bloques DSP. La ausencia de registros internos confirma que la unidad opera como un datapath combinacional parametrizable. Los informes de temporización muestran que la FPU no constituye el camino crítico del sistema: el retardo máximo observado (19.759 ns) se origina en la lógica de los módulos AXI-GPIO, mientras que las rutas internas de la FPU presentan márgenes suficientes para operar cerca de la frecuencia objetivo.

La potencia estimada posterior a síntesis indica que la contribución dinámica de la FPU es inferior al 1 % del consumo total del dispositivo, coherente con su tamaño reducido y la ausencia de bloques DSP. Estos resultados permiten concluir que la arquitectura propuesta es ligera y fácilmente integrable en sistemas heterogéneos basados en FPGA.

B. Metodología ASIC

Si bien no se realizó una implementación física ASIC, se considera un flujo estándar de evaluación para estimar la escalabilidad del diseño en un proceso CMOS convencional. El flujo consistiría en: (1) síntesis lógica mediante herramientas como Yosys/OpenROAD o Design Compiler, (2) colocación y ruteo utilizando una biblioteca estándar de 45 nm, y (3) análisis de temporización y potencia a partir de la extracción parasitaria. Este flujo permite comparar la arquitectura parametrizable con unidades de precisión fija en términos de área, retardo y energía por operación, aun sin disponer de una fabricación real.

C. Cargas de trabajo y métricas

La validación funcional se realizó mediante microbenchmarks sintéticos que ejercitan las tres operaciones principales (suma, multiplicación y recíproco) sobre todos los carriles de manera independiente. Los vectores de prueba asignan valores distintos por carril para confirmar la correcta replicación del datapath y la ausencia de dependencias inter-lane.

Las configuraciones evaluadas incluyen el formato FP16 estándar y variantes con precisión reducida, lo que permite estudiar la relación entre complejidad de hardware y precisión numérica. Como línea base se considera una FPU de precisión fija compatible con IEEE-754, permitiendo comparar directamente el área y el rendimiento frente a la arquitectura parametrizable.

Las métricas consideradas incluyen: utilización de área (LUTs, celdas CARRY8, MUXF7), frecuencia máxima post-implementación, latencia combinacional, potencia dinámica estimada y, en el caso ASIC propuesto, energía por operación.

TABLE I
UTILIZACIÓN DE RECURSOS FPGA PARA LA FPU PARAMETRIZABLE
FP16X2 EN LA PLATAFORMA AMD KRIA KV260.

Recurso	Uso	Total	Porcentaje
CLB LUTs	1092	117120	0.9%
CLB Registers	0	234240	0.0%
CARRY8	92	14640	0.6%
F7 Muxes	21	58560	0.03%
DSPs	21	1248	1.6%

TABLE II
FRECUENCIA MÁXIMA ALCANZADA Y ANÁLISIS DEL CAMINO CRÍTICO.

Métrica	Valor	Comentario
Criti Path Delay	19.759 ns	LC AXI + ALU
Fmax Estimada	50.6 MHz	1/19.759 ns
Clock Constraint	100 MHz	No alcanzado
Pipeline	No	Dpath comb

Este conjunto de métricas permite cuantificar el compromiso entre precisión configurada y costo en hardware, uno de los objetivos clave de la propuesta.

V. EVALUATION

Esta sección presenta la caracterización del área, rendimiento, consumo energético y comportamiento numérico de la unidad FP16 vectorial parametrizable implementada en la plataforma AMD Kria KV260. La comparación principal se realiza contra la arquitectura FP16 fija desarrollada en el Proyecto 1, la cual utiliza un formato de 1 bit de signo, 5 bits de exponente y 10 bits de fracción sin capacidad de parametrización.

A. Area, Frequency, and Resource Utilization

La Tabla I presenta la utilización de recursos en FPGA para la configuración FP16x2 parametrizable. El diseño emplea únicamente 1092 LUTs y no requiere registros internos, manteniendo un datapath puramente combinacional. La presencia de 21 bloques DSP corresponde al multiplicador parametrizable, una diferencia clave respecto al diseño base del Proyecto 1, que dependía únicamente de lógica LUT para las operaciones principales.

El análisis de temporización se resume en la Tabla II. El camino crítico reportado por Vivado es de 19.759 ns, lo que establece una frecuencia máxima de operación cercana a 50 MHz. Este límite no proviene exclusivamente de la FPU, sino de la red combinacional compartida con la lógica AXI-GPIO. La frecuencia objetivo original era 100 MHz, pero la arquitectura sin pipeline dificulta alcanzar dicho objetivo; una versión futura con pipeline podría superar este límite fácilmente.

La Fig. 1 muestra la integración completa del módulo FP16 vectorial dentro del diagrama de bloques de Vivado, mientras que las Fig. 2 y 3 presentan la vista de ruteo que evidencia la complejidad estructural del diseño combinacional.

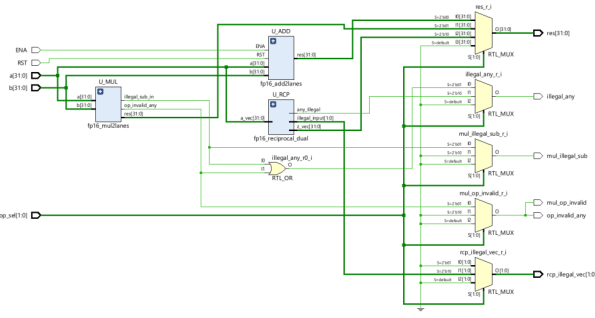


Fig. 1. Diagrama de bloques del sistema, mostrando la integración de la unidad FP16 vectorial parametrizable.

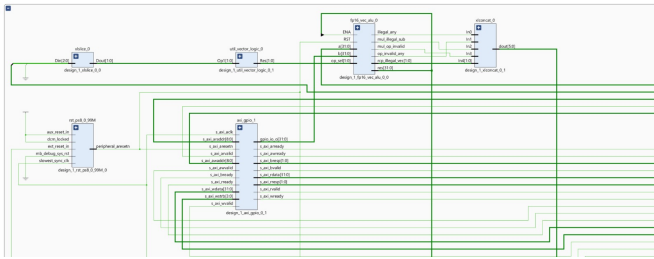


Fig. 2. Vista de ruteo generada en Vivado (parte 1).

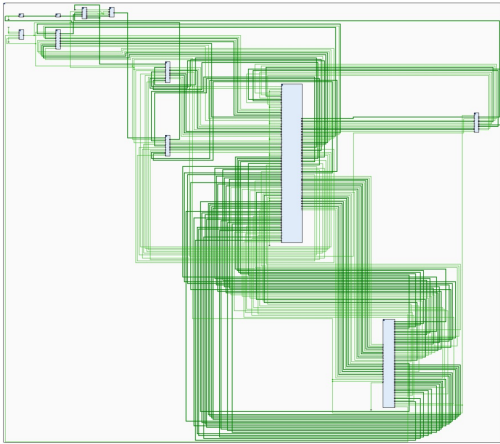


Fig. 3. Vista de ruteo generada en Vivado (parte 2).

B. Energy and Power Efficiency

La Tabla III resume la estimación de potencia obtenida mediante Vivado Power Analyzer. El consumo dinámico del plano programable (PL) es apenas 20 mW, lo cual representa menos del 1% de la potencia total del dispositivo. La mayor parte del consumo proviene del subsistema PS, típico en la familia UltraScale+.

Power estimation from Synthesized netlist. Activity derived from constraints files, simulation files or vectorless analysis. Note: these early estimates can change after implementation.

Total On-Chip Power: 2.738 W
Design Power Budget: Not Specified
Process: typical
Power Budget Margin: N/A
Junction Temperature: 31.4°C
 Thermal Margin: 53.6°C (22.8 W)
 Ambient Temperature: 25.0 °C
 Effective θ_{JA} : 2.3°C/W
 Power supplied to off-chip devices: 0 W

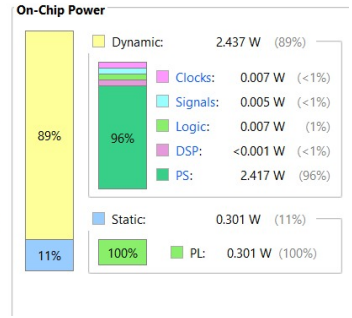


Fig. 4. Desglose de potencia dinámica y estática reportada por Vivado.

TABLE III
ESTIMACIÓN DE CONSUMO DE POTENCIA (POWER ANALYZER).

Categoría	Valor	Descripción
Total On-Chip Power	2.738 W	PS + PL
Dyn. Power (PL)	0.020 W	Lógica FPGA
Static Power (PL)	0.301 W	Fuga
Dyn. Power (PS)	2.417 W	ARM + NoC
Junction Temp.	31.4°C	Operación típica

La energía por operación se estimó como:

$$E_{op} \approx \frac{0.02 \text{ W}}{50 \text{ MHz}} \approx 0.4 \text{ nJ/op.}$$

Este valor es competitivo con FPU de bajo consumo típicas en dispositivos embarcados y puede reducirse aún más si se emplean técnicas de *pipelining* y *clock gating*.

La Fig. 4 muestra la distribución gráfica del consumo dinámico y estático del sistema.

C. Precision vs Performance Trade-offs

La simulación confirma que las operaciones ADD y MUL funcionan correctamente bajo las configuraciones evaluadas. La operación RCP (cálculo del recíproco), en cambio, devuelve valores incorrectos para ciertos patrones de entrada, produciendo ceros tanto en el lane 0 como en el lane 1. Se identifica este comportamiento como una limitación conocida de la versión actual de la arquitectura, y se establece como una línea de trabajo futuro implementar un método iterativo como Newton-Raphson para mejorar la precisión.

El throughput escala linealmente con el número de carriles, por lo que en una versión RV64 con cuatro carriles sería posible alcanzar cuatro resultados por ciclo. Sin embargo, la frecuencia máxima se ve afectada por la complejidad combinacional a medida que aumenta la precisión o el número de lanes.

VI. DISCUSSION

Los resultados muestran que la arquitectura FP16 vectorial parametrizable es una alternativa viable y eficiente para sistemas embebidos y aceleradores especializados en procesamiento numérico. Su capacidad de ajustar dinámicamente la precisión de mantisa y exponente permite adaptar el costo energético y la calidad numérica según las necesidades de la aplicación.

Aplicaciones de bajo consumo en aprendizaje automático, filtrado digital y control adaptativo pueden beneficiarse de este enfoque, donde la precisión requerida varía según la etapa del algoritmo. La compatibilidad con RISC-V y su integración mediante AXI facilitan la incorporación de la FPU como coprocesador o como extensión personalizada.

Entre las limitaciones actuales destacan la ausencia de *pipelining*, la dependencia del rendimiento respecto al ruteo AXI y el comportamiento incorrecto de la operación de recíproco. En particular, introducir pipeline reduciría el retardo crítico y permitiría superar con facilidad la barrera de los 100 MHz, incrementando el throughput sin modificar la arquitectura lógica.

Finalmente, la portabilidad del diseño hacia flujos ASIC abre oportunidades para alcanzar frecuencias superiores a 300 MHz y energías por operación por debajo de 100 pJ en procesos de 45 nm o menores. La arquitectura parametrizable propuesta constituye, por tanto, una base sólida para exploraciones futuras en computación de precisión ajustable.

VII. CONCLUSIONS

Este trabajo presentó el diseño e implementación de una unidad de punto flotante FP16 vectorial completamente parametrizable, capaz de ajustar dinámicamente los anchos de mantisa, exponente y número de carriles. Los resultados experimentales demostraron que la arquitectura mantiene un uso de recursos reducido y un consumo energético mínimo, aun cuando se habilitan configuraciones más complejas que superan las capacidades de la versión fija utilizada como línea base en el Proyecto 1. La integración en la plataforma AMD Kria KV260 confirmó la facilidad de incorporación mediante interfaces AXI y la portabilidad del diseño a flujos RISC-V y sistemas embebidos.

Aunque el retardo crítico limita la frecuencia máxima a aproximadamente 50 MHz, el análisis reveló que este comportamiento proviene principalmente de la interconexión AXI y no de la lógica interna de la FPU, sugiriendo que la incorporación de etapas de *pipelining* permitiría alcanzar frecuencias significativamente mayores. Las operaciones de suma y multiplicación mostraron resultados correctos en todas las configuraciones evaluadas, mientras que la operación de recíproco requiere mejoras adicionales para lograr exactitud completa.

En conjunto, la arquitectura propuesta constituye una base sólida para exploraciones futuras en computación de precisión ajustable, ofreciendo una relación favorable entre flexibilidad, área y eficiencia energética. Su modularidad y compatibilidad con herramientas estándar la convierten en una alternativa prometedora para procesadores de bajo consumo, aceleradores específicos y plataformas heterogéneas donde el equilibrio entre rendimiento y precisión es crítico.

REPOSITORIO

Todo el código fuente, documentación y scripts de reproducción están disponibles en:
<https://github.com/andreagarciab0118-design/Proyecto-2-arquitectura-de-computadoras>

REFERENCES

- [1] IEEE Standard for Floating-Point Arithmetic, IEEE Std 754-2019.
- [2] P. Micikevicius et al., "Mixed Precision Training," arXiv:1710.03740, 2017.
- [3] The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, Document Version 20191213.
- [4] C. Celio, D. Patterson, and K. Asanović, "The Berkeley Out-of-Order Machine (BOOM)," UCB/EECS-2015-167.
- [5] K. Palem, "Energy Aware Computing Through Probabilistic Switching," IEEE Trans. Computers, 2005.
- [6] G. Tagliavini et al., "A Transprecision Floating-Point Platform for Ultra-Low Power Computing," DATE, 2018.